

CST8507: NATURAL LANGUAGE PROCESSING

WEEK#13
**LLM COMPRESSION
&
PROMPT ENGINEERING**

DEVELOPED BY
HALA OWN, PH.D.

Lesson Agenda

- Final Exam
- Model Compression techniques
 - Introduction to Prompt Engineering
 - Why it is important
 - Benefits of Prompt Engineering
 - Types of Prompt Engineering
 - Prompt Engineering best practice



Final Exam

- Final test duration is 120 minutes
 - When: Monday 20th April 2026
 - Start: 12:30 pm
 - Where: WB 384A
- Closed book exam (**one-page, double-sided allowed, Please make sure to leave a 5 cm by 5 cm space in the top-left corner of each side of your cheat sheet for the proctor's signature.**)
- 40 questions – MC and True/False (**1 POINT FOR EACH QUESTION**)
- 6 questions – answers (**5 POINTS FOR EACH QUESTION**)



FINAL EXAM MARKS

- Final exam marks **will not be posted** on Brightspace.
Final letter grades will be available on ACSIS once they have been approved by the Chair. After this approval, your final exam mark will be released on Brightspace.



How to Prepare

- Lecture summary slides are a good place to start:
 - they don't have all the details, but make sure you understand the details underlying the main points mentioned.
- Do the labs! Make sure you understand the answers you get.
- Code-Examples demonstrated during the lecture (check lecture materials folder on Brightspace).
- Hybrid work
- Class Activities



Comparison of Popular Large Language Models



Comparison of Popular Large Language Models

Model	Parameters	Size on Disk	Memory Usage (Inference)	Learning Data Size
BERT (Large)	340M	~1.3 GB (FP32)	~1.5–2 GB (FP16)	3.3B words (~16 GB)
GPT-4o	~200B	~350 GB (FP32)	~400 GB (FP16, single GPU)	570 GB (~300B tokens)
LLaMA (13B)	13B	~26 GB (FP32)	~26 GB (FP16)	~1T tokens
LLaMA (70B)	70B	~140 GB (FP32)	~140 GB (FP16)	~1T tokens
BLOOM (176B)	176B	~352 GB (FP32)	~352 GB (FP16)	1.6T tokens
Mistral 7B	7B	~14 GB (FP32)	~14 GB (FP16)	~1T tokens
Mixtral 8x7B	56B	~112 GB (FP32)	~112 GB (FP16)	Unknown (large corpus)
Grok (xAI)	Unknown (est. ~70B)	Est. ~140 GB (FP32)	Est. ~140 GB (FP16)	Unknown (large)
PaLM (540B)	540B	~1 TB (FP32)	~1 TB (FP16)	780B tokens

Real Life Example

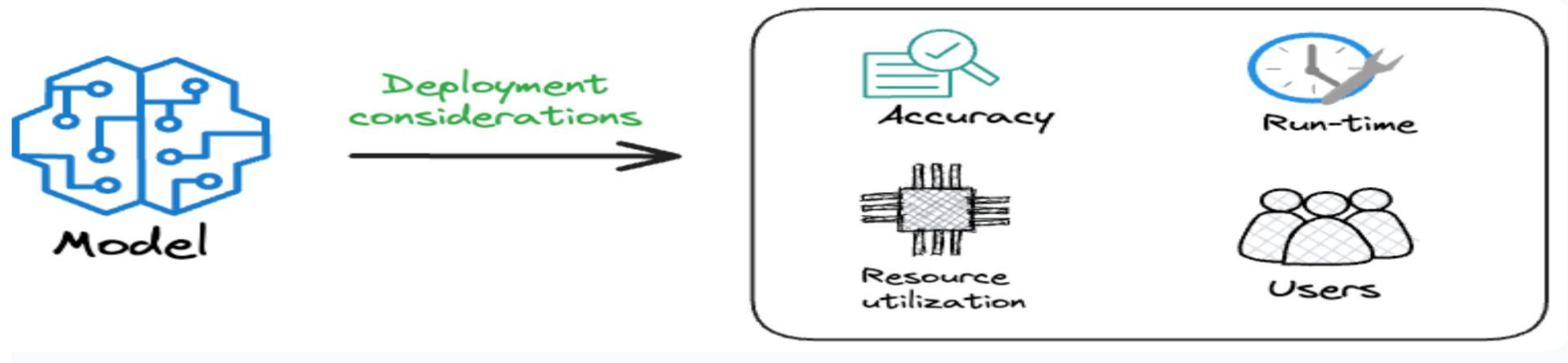


CASEY JOHNSTON

BUSINESS APR 16, 2012 8:28 AM

Netflix Never Used Its \$1 Million Algorithm Due To Engineering Costs

Netflix awarded a \$1 million prize to a developer team in 2009 for an algorithm that increased the accuracy of the company's recommendation engine by 10 percent. But it doesn't use the million-dollar code, and has no plans to implement it in the future, Netflix announced on its blog Friday. The post goes on to explain why: [...]



<https://www.wired.com/2012/04/netflix-prize-costs/?ref=dailydoseofds.com>

Discussion



Model A

Accuracy: 99%

Run-time: 2 seconds

Size: 125 MBs



Model B

Accuracy: 97%

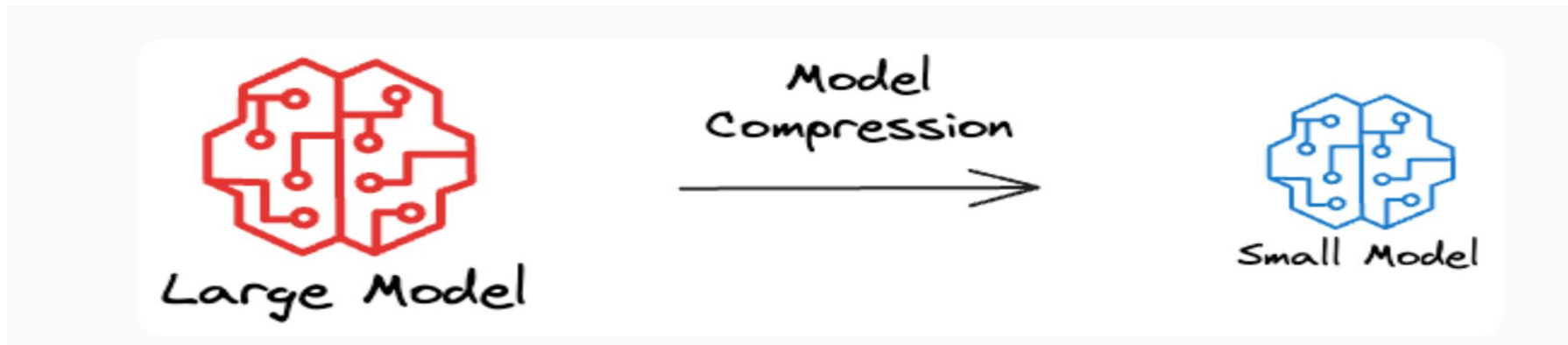
Run-time: 0.1 seconds

Size: 10 MBs



Motivation

What approaches do you think can help us deploy NLP systems in a way that is **cost effective**, **efficient**, and **equitable** without a **significant loss in accuracy**.?



Answer: Model Compression

Model Compression

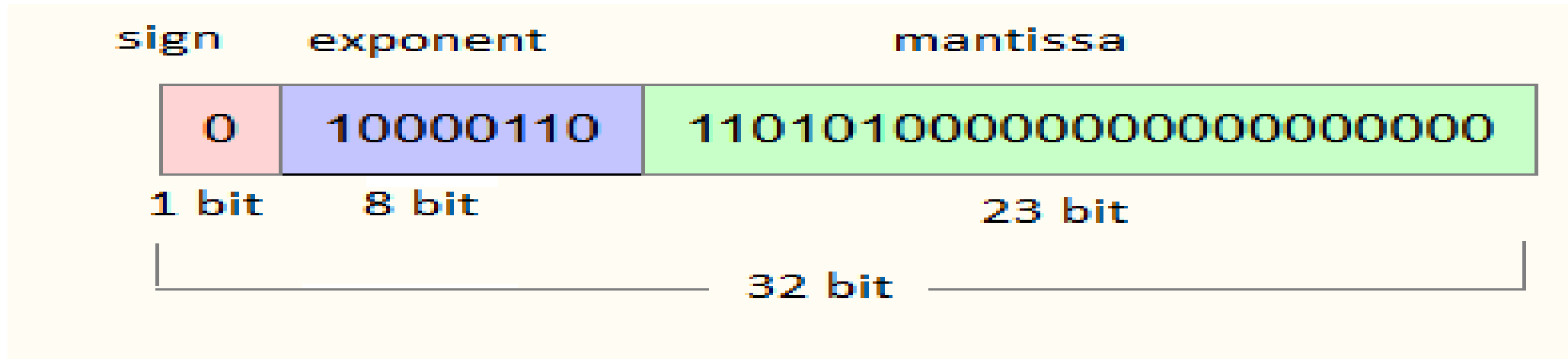
- Quantization
- Pruning
- Knowledge Distillation



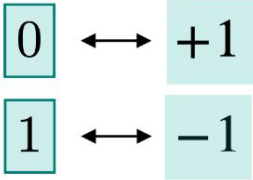
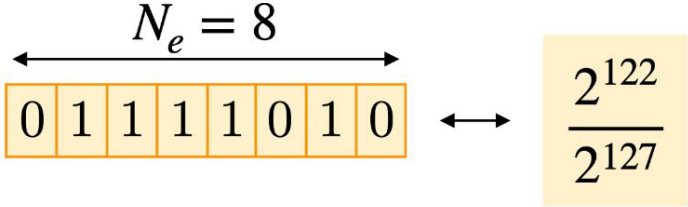
QUANTIZATION



Floating Point Presentation



Floating Point Presentation...

Name	Description	Illustration
Sign	Controls whether the number is positive or negative. Typically takes up to 1 bit.	 0 ↔ +1 1 ↔ -1
Exponent	Controls the magnitude of the number. Also called <i>range</i> .	 $N_e = 8$ 0 1 1 1 1 0 1 0 ↔ $\frac{2^{122}}{2^{127}}$
Mantissa	Controls the granularity of the number, i.e. what is after the decimal point. Also called <i>significand</i> or <i>fraction</i> .	 1 1 0 0 0 ... ↔ 1.75

Floating Point Presentation...

	Sign	Exponent	Mantissa
FP16 (Floating-Point 16)	1	5	10
FP32 (Floating-Point 32)	1	8	23
FP64 (Floating-Point 64)	1	11	52

Precision of Numbers

1.2015432...

2.7015402...

2.4024402...

-0.7055120...

-1.7067140...

0.2741131...

-1.5312410...

0.4025222...



Precision of Numbers

1.2015432...

2.7015402...

2.4024402...

-0.7055120...

-1.7067140...

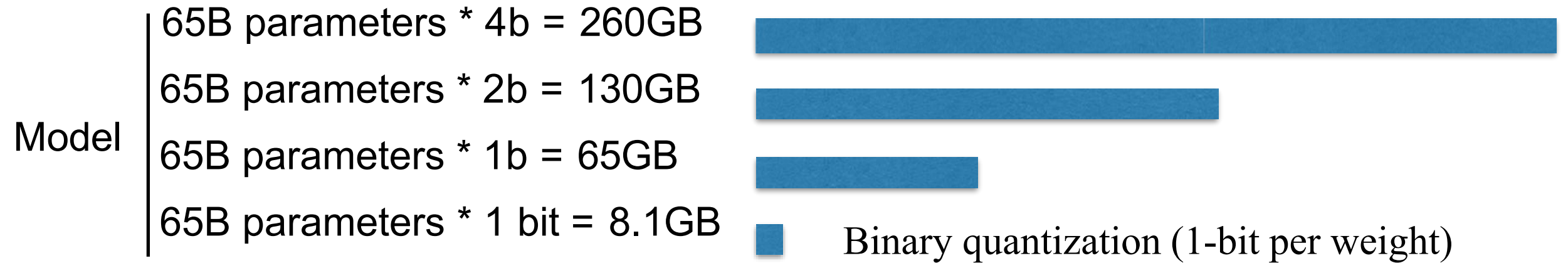
0.2741131...

-1.5312410...

0.4025222...



Quantization



Converts model weights from **32-bit floating point (FP32)** to **lower precision (e.g., INT8, FP16)**.



Quantization: Computational Performance of a GPU under Different Numerical Precisions

FP64	9.7 TFLOPS
FP64 Tensor Core	19.5 TFLOPS
FP32	19.5 TFLOPS
Tensor Float 32 (TF32)	156 TFLOPS 312 TFLOPS*
BFLOAT16 Tensor Core	312 TFLOPS 624 TFLOPS*
FP16 Tensor Core	312 TFLOPS 624 TFLOPS*

Lower precision → **Faster** processing



```
model = GPT2LMHeadModel.from_pretrained("gpt2")
tokenizer = GPT2Tokenizer.from_pretrained("gpt2")

# Convert to quantization
model.eval()
quantized_model = torch.quantization.quantize_dynamic(
    model, {torch.nn.Linear}, dtype=torch.qint8
)

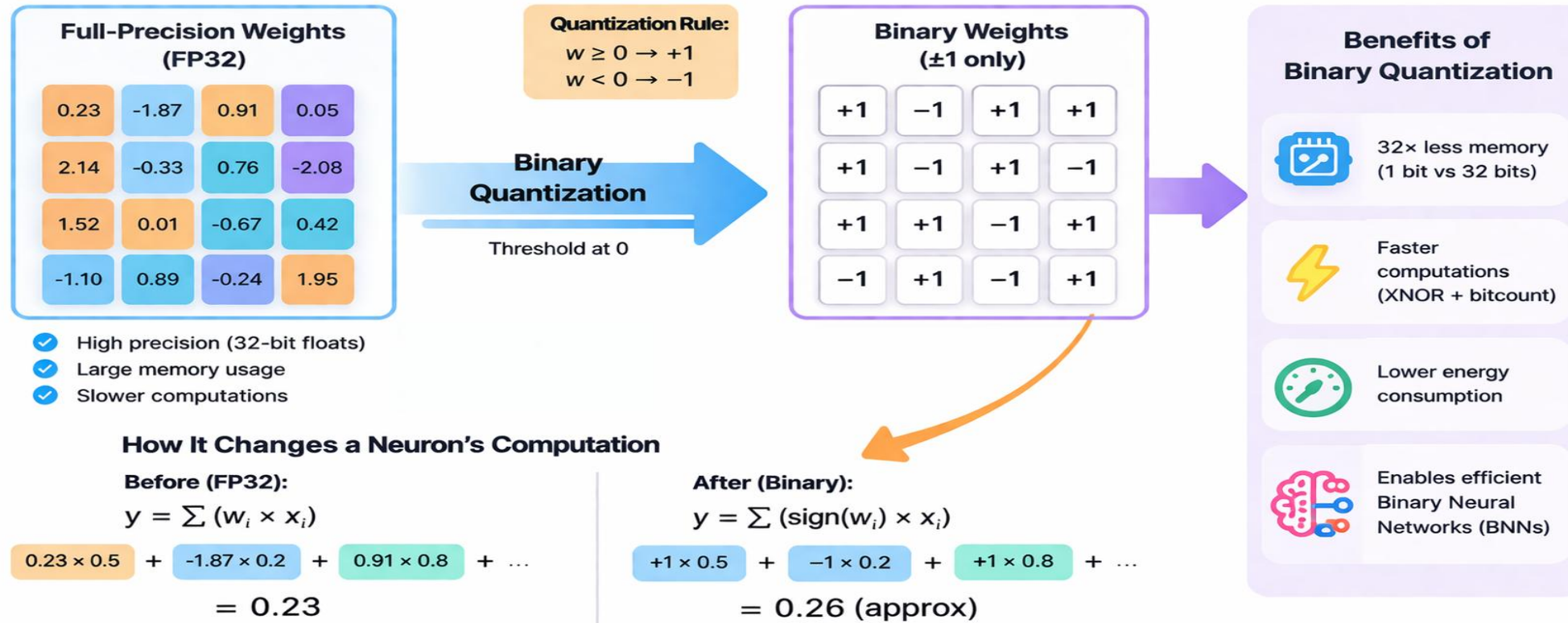
# Save the quantized model
torch.save(quantized_model.state_dict(), "quantized_gpt2.pth")

with torch.no_grad():
    output = quantized_model.generate(**inputs, max_length=50)
```



Binarized Neural Networks

Binary Quantization in Neural Networks (BNN)



Example: Microsoft's BitNet



Research

Our research ▾

Programs & events ▾

Connect & learn ▾

About ▾

Register: Research Forum

The Era of 1-bit LLMs: All Large Language Models are in 1.58 Bits

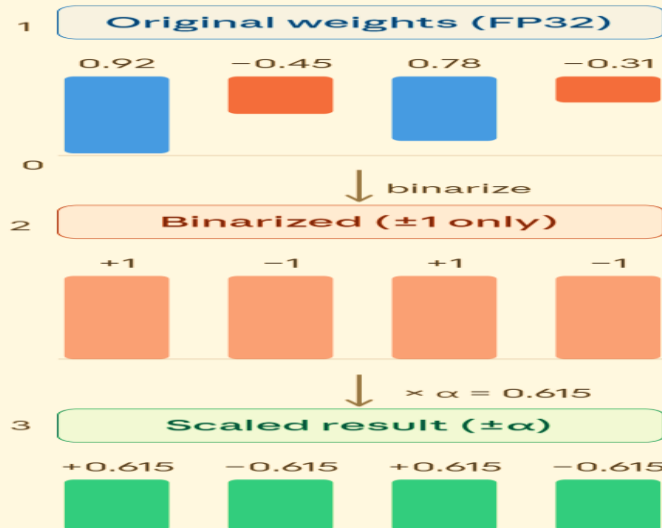
Shuming Ma, Hongyu Wang, Lingxiao Ma, Lei Wang, Wenhui Wang, [Shaohan Huang](#), Lifeng Dong, Ruiping Wang, Jilong Xue, [Furu Wei](#)

February 2024

arXiv

Work in progress

[Publication](#) | [Publication](#)



Scaling factor α

$$\begin{aligned}\alpha &= \text{mean}(|W|) \\ &= (0.92 + 0.45 + 0.78 + 0.31) / 4 \\ &= 2.46 / 4 = 0.615\end{aligned}$$

Key formula

$$W \approx \alpha \times B$$

W = original FP32 weights

B = binarized matrix (± 1)

α = $\text{mean}(|W|)$ — one FP32 scalar

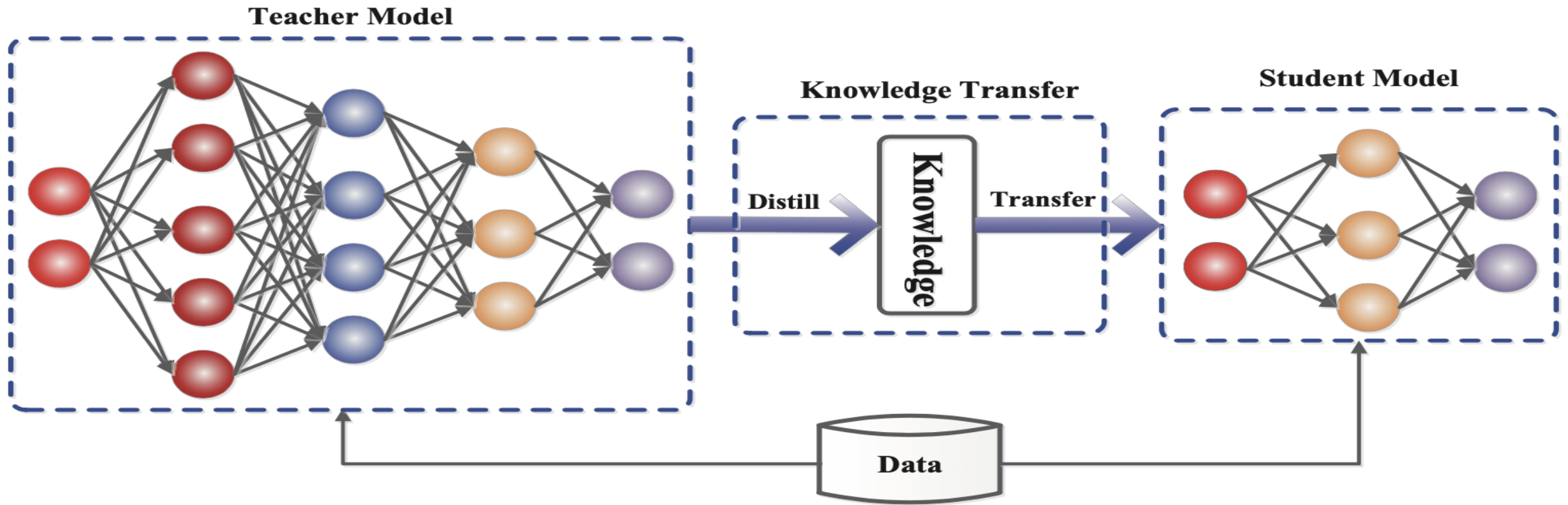


DISTILLATION



Distillation

Transferring knowledge from a **large, complex model** (the **teacher model**) to a smaller, more efficient model (the **student model**)



Distillation Loss Function

$$L = \alpha \cdot L_{soft} + \beta \cdot L_{hard}$$

Student Loss (L_{hard})

Cross-Entropy Loss between the true labels, and the student model's predictions

Distillation Loss (L_{soft})

KL Divergence between the **teacher's soft predictions** and the **student model's predictions**

- α and β balance the loss terms



Example of Distilled Models

Model	Teacher Model	Student Model	Model Size Reduction	Inference Speed Improvement	Performance Retained	Use Case
DistilBERT	BERT	DistilBERT	60%	2x	97%	Real-time, mobile applications
DistilGPT-2	GPT-2	DistilGPT-2	60%	2x	97%	Text generation, chatbots
Distilled T5	T5	Distilled T5	60%	Faster than T5	96%	Translation, summarization, Q&A



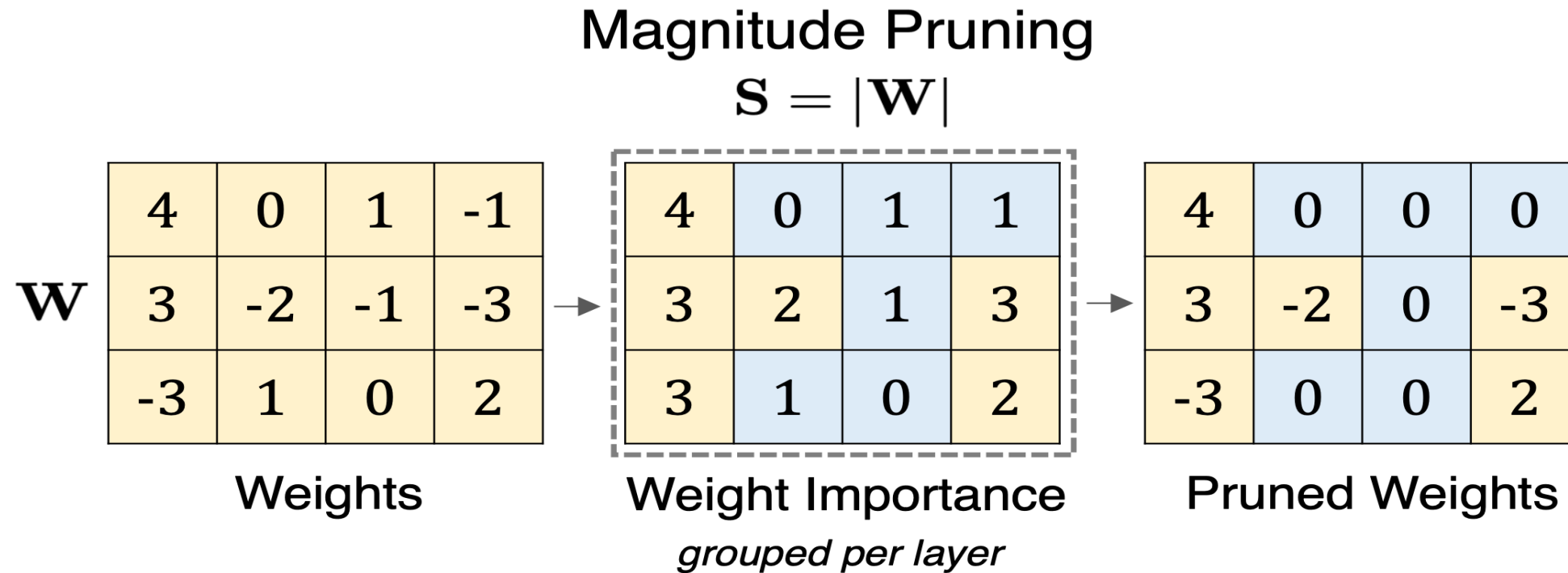
PRUNING



Pruning

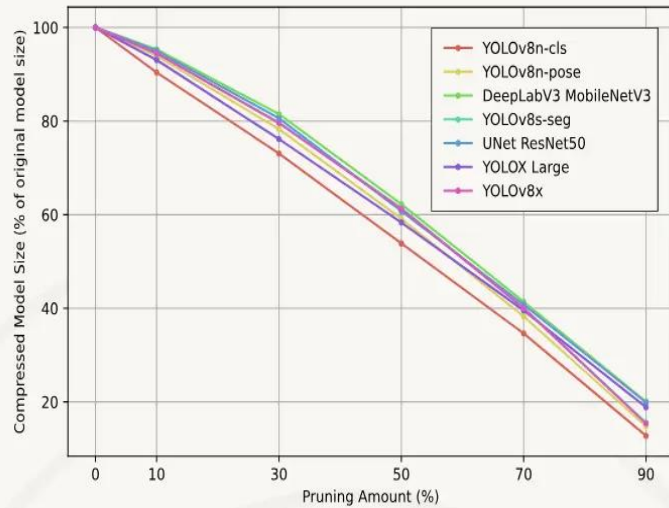
Magnitude-Based Weight Pruning

- Remove parameters from the model after training



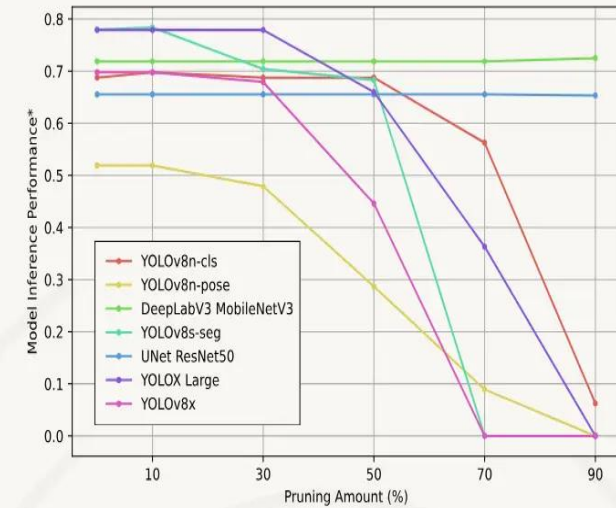
Empirical Effects of Pruning

Datature | Blog



COMPRESSED MODEL SIZE REDUCTION
ACROSS VARIOUS PRUNING RATIOS

Datature | Blog



MODEL INFERENCE PERFORMANCE*
ACROSS VARIOUS PRUNING RATIOS



Distillation vs Quantization vs Pruning

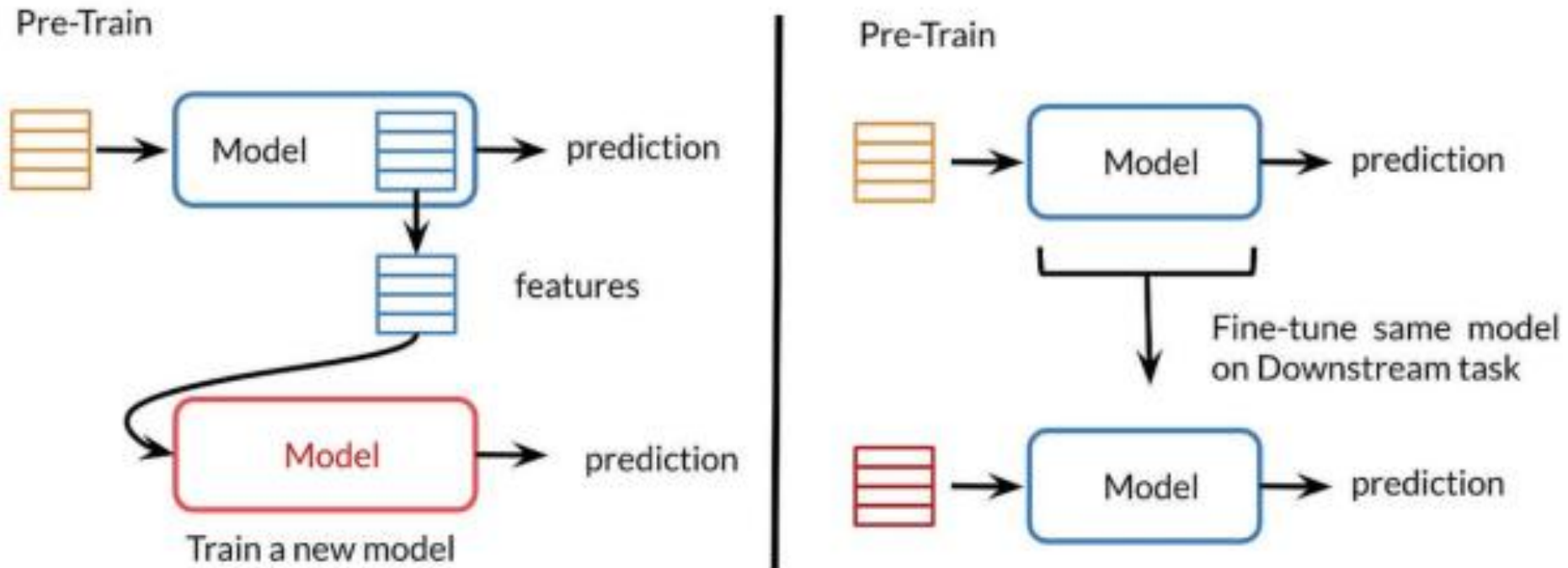
Quantization: no parameters are changed, up to k bits of precision

Pruning: a number of parameters are set to zero, the rest are unchanged

Distillation: all parameters are changed



Transfer Learning vs Fine Tuning LLMs



Supervised Fine-Tuning

Model	SFT size (# examples)
GPT-3	13 thousands
LLaMA 3	10 million

SFT = Supervised FineTuning

How Supervised Fine-Tuning Works

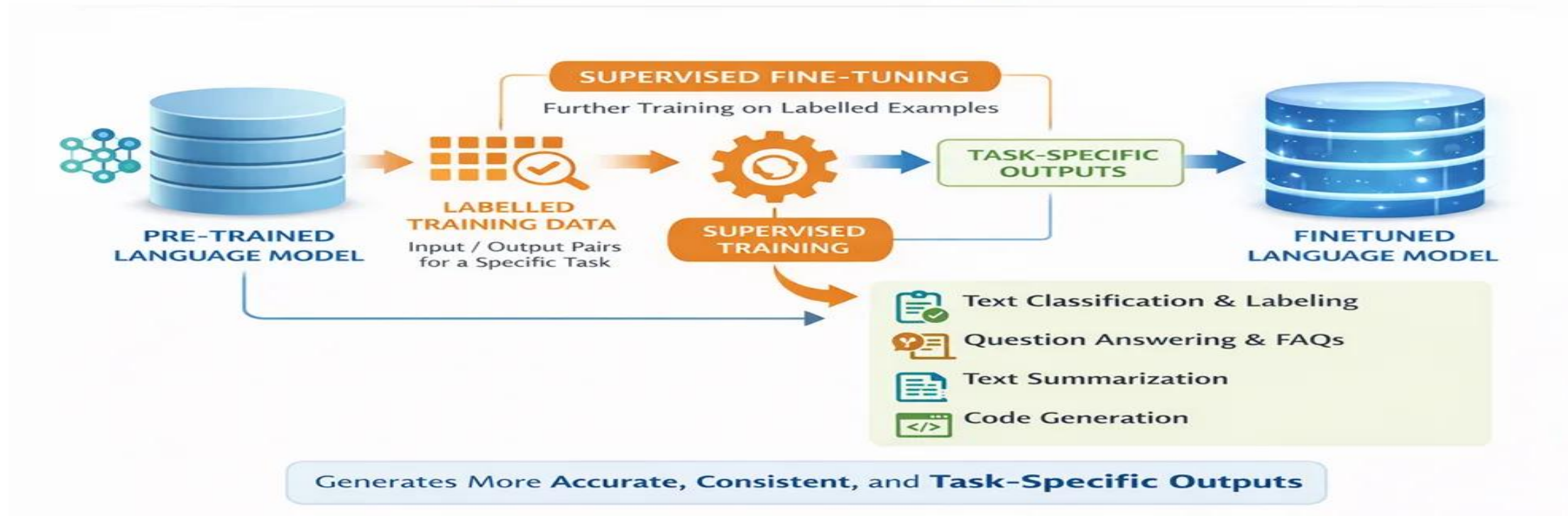


Image source: <https://devblogs.microsoft.com/foundry/beyond-the-prompt-why-and-how-to-fine-tune-your-own-models/>

Problem Fine-Tuning

- ❑ Large models → Billions of parameters
- ❑ GPU memory constraints
- ❑ Computational costs.
- ❑ Very high-quality data needed



Parameter-efficient Fine-tuning (PEFT)

Don't tune all of the parameters, but just some!

- Prompt/prefix
- Adapters
- BitFit
- **Low-Rank Adaptation technique (LoRA)**



What is LoRA

Low-rank adaptation technique that reduces fine-tuning costs.

□ Main idea:

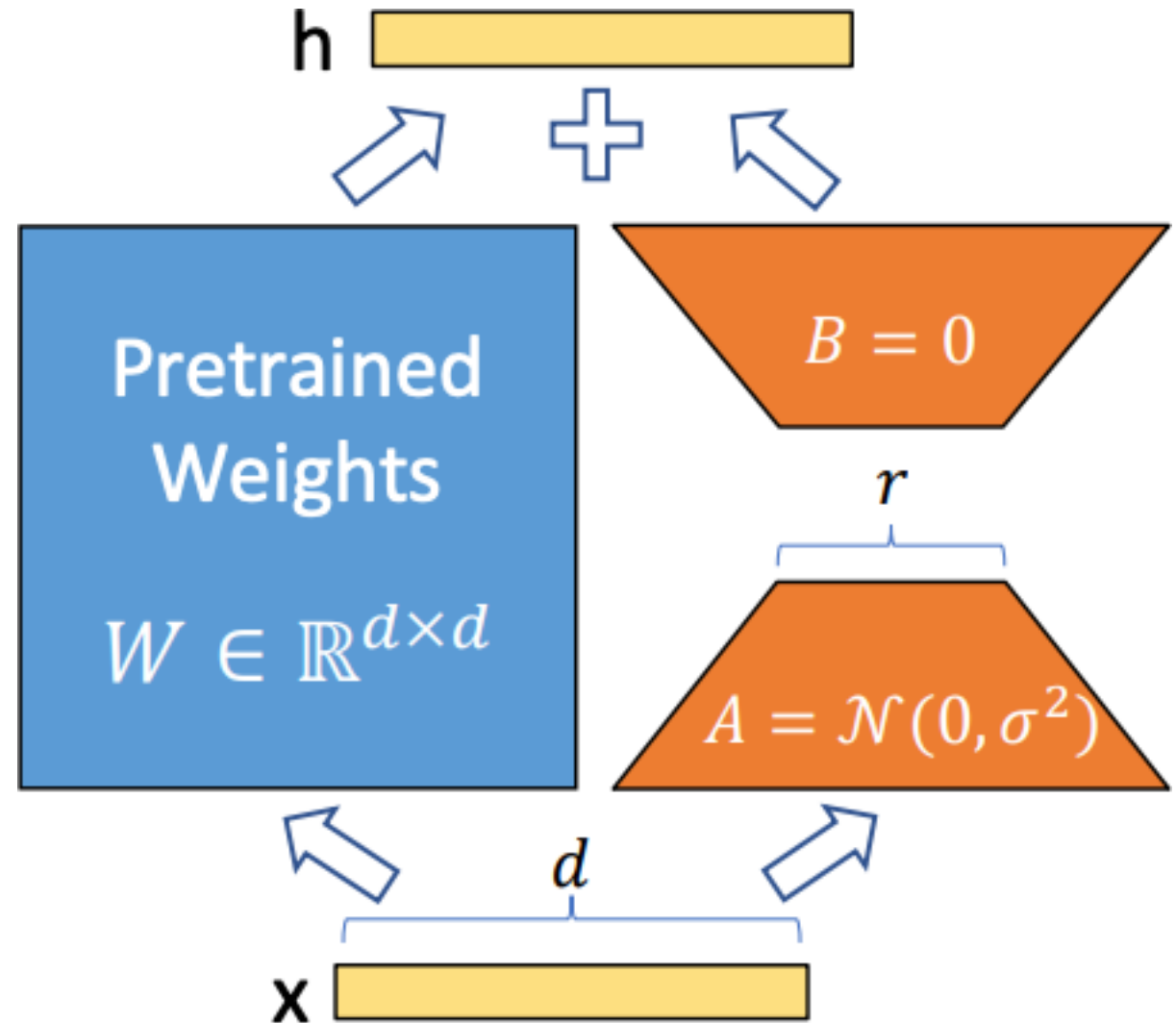
- Decomposes weight updates into two smaller low-rank matrices (A & B).
- Reduces trainable parameters while keeping model quality high.



LoRA (Hu et al. 2021)

Freeze pre-trained weights, train low-rank approximation of difference from pre-trained weights

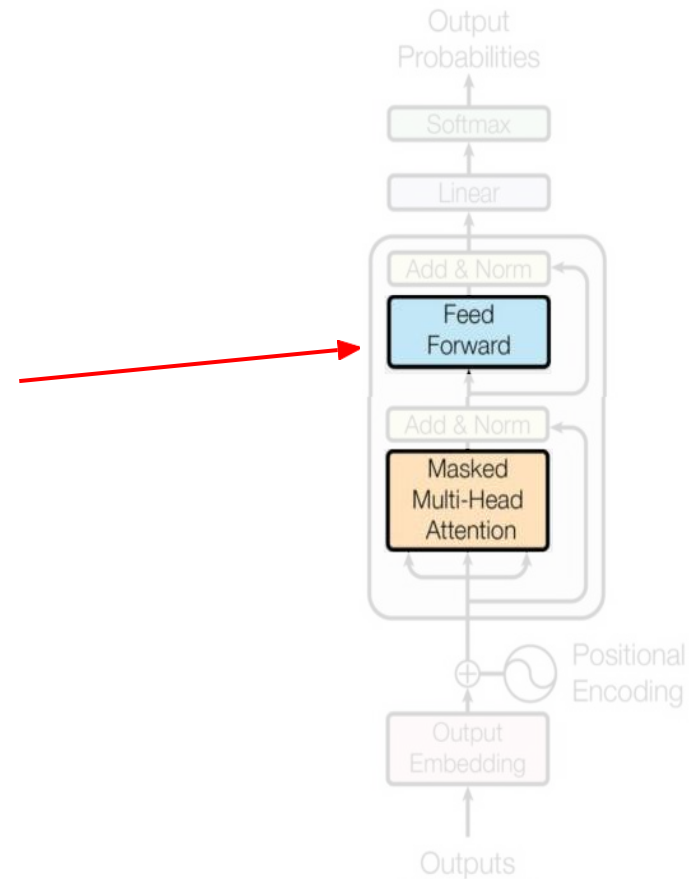
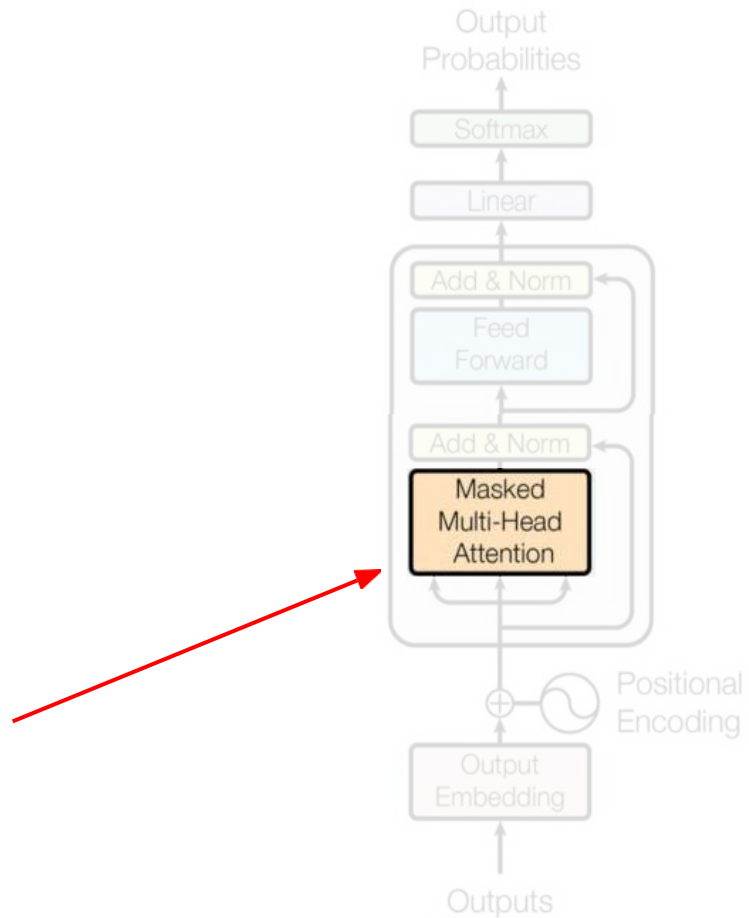
Advantage: After training, just add in to pre-trained weights — no new components!



Only A and B contain **trainable** parameters



Where to Apply LoRA



Empirical Facts

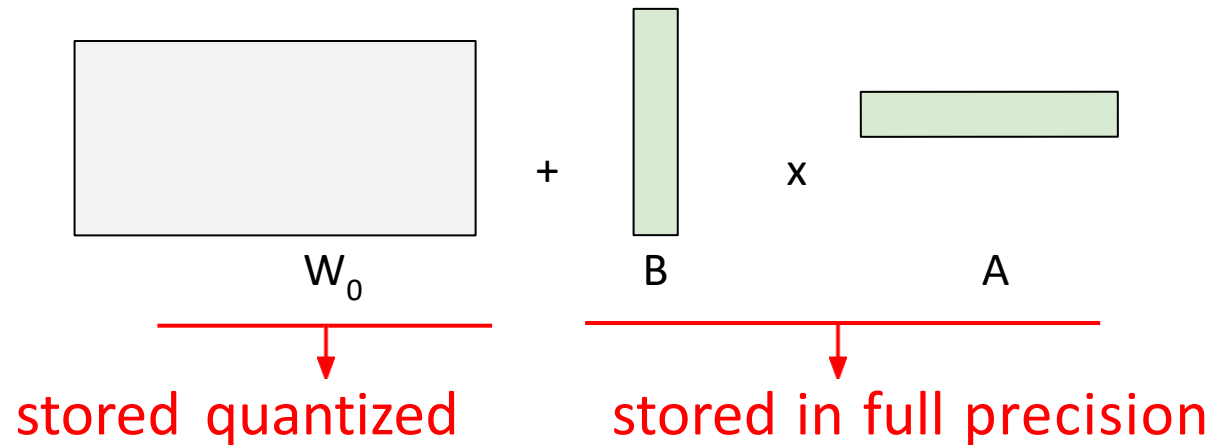
"LoRA Without Regret", Schulman et al., 2025..

- LoRA needs a **higher learning rate** than full fine-tuning
- LoRA does **poorly on large batch size** compared to full fine-tuning



Q-LORA (Dettmers et al. 2023)

Quantize all frozen weights to relieve memory bottleneck.



4-bit quantization of the model Use of GPU

Can **train a 65B** model on a **48GB GPU**!



PROMPT ENGINEERING



Prompt Engineering

- The art of **asking the right question** to get the **best output** from an LLM. It enables direct interaction with the LLM using only plain language prompts.
- **Prompts** involve instructions and context passed to a language model to achieve a desired task

What is prompt engineering?

Prompt engineering is a process of creating a set of prompts, or questions, that are used to guide the user toward a desired outcome. It is an effective tool for designers to create user experiences that are easy to use and intuitive. This method is often used in interactive design and software development, as it allows users to easily understand how to interact with a system or product..



Elements of a Prompt

- A prompt is composed with the following components:

- Context
- Instructions
- Input data
- Output indicator

You are a data scientist working on a sentiment analysis, **classify the text into pos , neg and neu**
Text: I think the food was ok
Sentiment:

Benefits of Prompt Engineering

- Improved task performance
- Controlling output
- Improving response quality
- Enhancing the interpretability
- Bias mitigation



Prompting Techniques



Zero-shot Prompting



Few-shot Prompting



Chain-of-Thought Prompting



Meta Prompting



Self-Consistency



Generate Knowledge Prompting



Prompt Chaining



Tree of Thoughts



Retrieval Augmented Generation



Automatic Reasoning and Tool-use



Automatic Prompt Engineer



Active-Prompt



Directional Stimulus Prompting



Program-Aided Language Models



ReAct



Reflexion



Multimodal CoT



Graph Prompting

Common Types of Prompt Engineering

- Example based prompt(Zero shot , One shot and Few-shot)
- Instruction based prompt
- Chain of thought COT
- Role-Based Prompting
- Persona-Guided Prompting



Example Based Prompt

Zero-shot prompt

Prompting without examples

Classify the text into neutral, negative, or positive.

Text: I think the food was okay.
Sentiment: ...

One-shot prompt

Prompting with a single example

Classify the text into neutral, negative, or positive.

Text: I think the food was alright.
Sentiment: **Neutral**

Text: I think the food was okay.
Sentiment:

Few-shot prompt

Prompting with more than one example

Classify the text into neutral, negative, or positive.

Text: I think the food was alright.
Sentiment: **Neutral**.

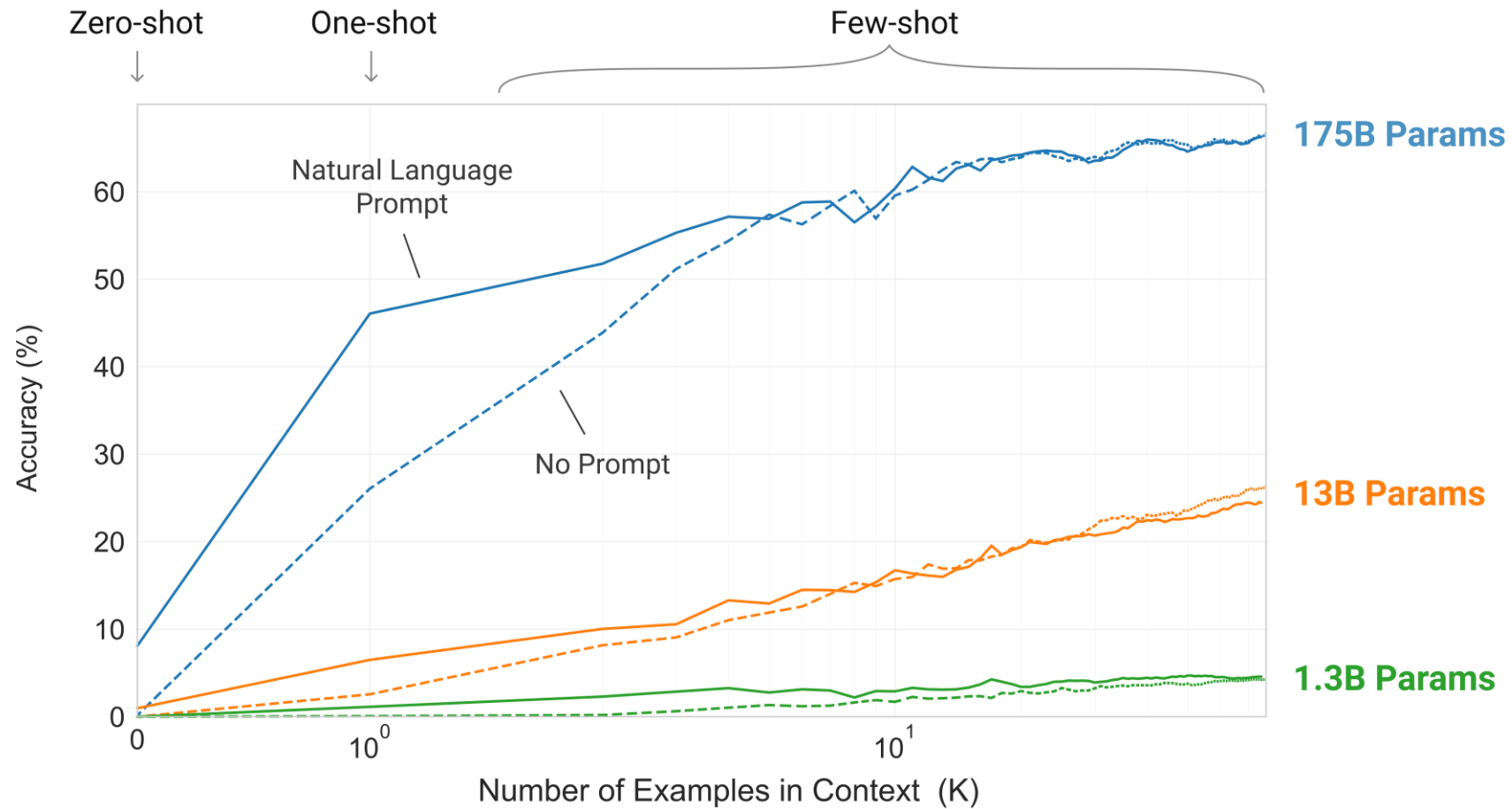
Text: I think the food was great!
Sentiment: **Positive**.

Text: I think the food was horrible...
Sentiment: **Negative**.

Text: I think the food was okay.
Sentiment:



GPT-3 (Generative Pretrained Transformer)



Source: Language Models are Few-Shot Learners

Chain Of Thought COT

One-shot prompt

Prompting with a single example

Q: Roger has 5 tennis balls. He buys 2 more cans of tennis balls. Each can has 3 tennis balls. How many tennis balls does he have now?

A: The answer is 11.

Q: The cafeteria had 23 apples. If they used 20 to make lunch and bought 6 more, how many apples do they have?

A: The answer is 27. ❌

Chain-of-thought prompt

Prompting with a **reasoning** example

Q: Roger has 5 tennis balls. He buys 2 more cans of tennis balls. Each can has 3 tennis balls. How many tennis balls does he have now?

A: Roger started with 5 balls. 2 cans of 3 tennis balls each is 6 tennis balls.
 $5 + 6 = 11$
The answer is 11.

Q: The cafeteria had 23 apples. If they used 20 to make lunch and bought 6 more, how many apples do they have?

A: The cafeteria had 23 apples originally. They used 20 to make lunch. So they had $23 - 20 = 3$. They bought 6 more apples, so they have $3 + 6 = 9$.
The answer is 9. ✅

Example

Reasoning process (thought)

Instruction

Reasoning process (thought)

Chain of thought COT

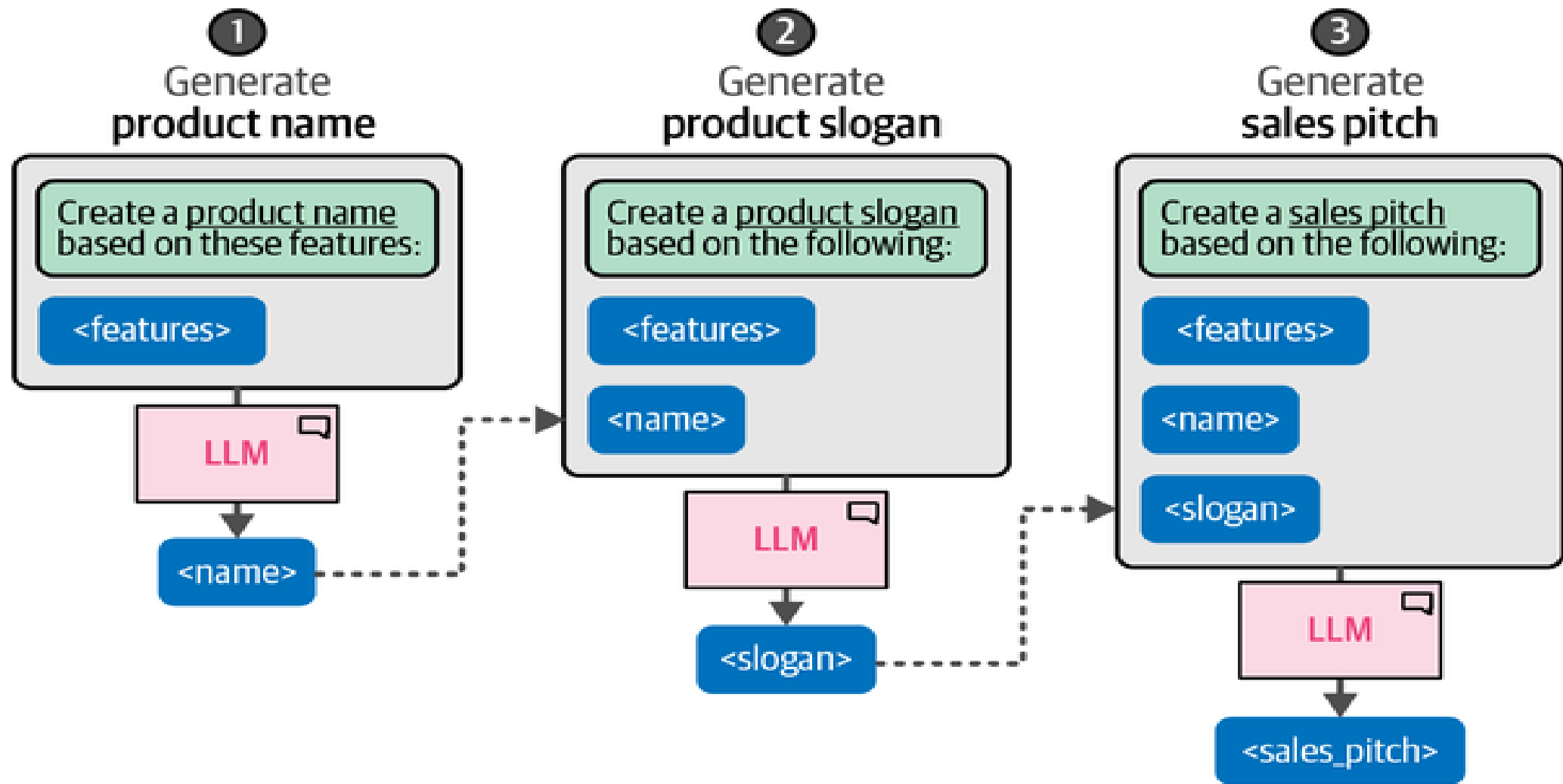
Standard prompt: *“If you have 5 apples and you give away 2, how many do you have left?”*

Chain of Thought Prompt: “You start with 5 apples. If you give away 2 apples, you subtract 2 from 5 to find out how many apples you have left. So, 5 minus 2 equals 3. Therefore, you have 3 apples left.”

Benefit: leads the **AI through a logical sequence of steps** to solve a simple arithmetic problem, making it easier to follow and verify the solution.



Chain Prompts



Role-Based Prompting

You are a hiring manager at a top tech company. Conduct a mock interview for a software developer position. Ask one question at a time and wait for my response before continuing.

Great! Let's get started with the interview.

Question 1:

"Can you please tell me about yourself and your background in software development?"



Persona-Guided Prompting

You are William Shakespeare. Write a poem about the beauty of the night.

Ode to the Night

Oh, velvet sky with silver'd glow,
Where quiet winds of midnight blow,
Thy stars like lanterns gently gleam,
Soft whisper'd light in slumber's dream.
The moon, a queen in pale embrace,
Doth kiss the earth with tender grace,
She walks 'tween clouds in silent flight,
A beacon fair to lovers' sight.



Prompt Templates

A template where you fill in with an actual input

Prompt & Prompt Template

Example code (e.g. Python)

1 prompt_template instead of prompt

```
prompt_template = """Act as support staff.  
Help the owners of the HHCR3000 operate their cleaning  
robot by giving answers to questions on features and step-  
by-step instructions when they ask for help.
```

User: **{query}** ——— **2** Variable in the template.
Assistant: """

```
# for each conversation turn  
prompt = prompt_template.format(query=actual_user_query)
```

3 Variable in the template is replaced by current user query to get the prompt

Prompting Best Practices

1. Be specific and clear.
2. Provide relevant context.
3. Break down tasks for complex problems.
4. Use chain-of-thought reasoning for problem-solving.
5. Experiment with few-shot or zero-shot prompting.
6. Use role-based or persona-guided prompts.
7. Ask for clarification if the response is unclear.
8. Be mindful of bias and ethical considerations.
9. Request structured responses when needed.
10. Iterate and refine the prompts based on responses.



Q&A

